

AXI DMA depuración

Referencia: AXI DMA v7.1 LogiCORE IP Product Guide

1. DMA Programming sequence

- Para iniciar **una operación MM2S (TX)** la secuencia es:

- 1) Start the MM2S channel haciendo DMACR.RS=1 (esto provoca que DMASR.Halted=0)
- 2) Opcional. Habilitar las interrupciones DMACR.IOC_IrqEn=1 y DMACR.Err_IrqEn=1
- 3) Escribir el source address en MM2S_SA
- 4) Escribir el nBytes a transferir en MM2S_LENGTH (**esto siempre tiene que ser lo último**)

- Para iniciar una **operación S2MM(RX)** la secuencia es:

- 1) Start the S2MM channel haciendo DMACR.RS=1 (esto provoca que DMASR.Halted=0)
- 2) Opcional. Habilitar las interrupciones DMACR.IOC_IrqEn=1 y DMACR.Err_IrqEn=1
- 3) Escribir el destination address en S2MM_DA
- 4) Escribir el nBytes a transferir en S2MM_LENGTH (**esto siempre tiene que ser lo último**)

2. DMA Registers

Nomenclatura RX/TX es desde el punto de vista del PS

DMA_TYPE_RX => S2MM offset (0x30)

DMA_TYPE_TX => MM2S offset (0x00)

DMACR: Control Register

- b0 RS: 0 stop, finaliza la operación en curso y para el DMA => provoca Halted =1
1 run, DMA preparada para iniciar transacciones => provoca Halted =0
- b2 Reset: 0 normal operation
1 reset in progress, finaliza la operación en curso y resetea todos los bits de los registros (a su estado de reset)
- b12 IOC_IrqEn: 0 int. disabled
1 int. enabled

DMASR: Status Register

- b0 Halted: 0 DMA channel running
1 DMA channel halted
- b1 Idle: 0 transfer is not completed (busy)
1 transfer completed y DMA paused

Inicialmente a 0 antes de iniciar una transferencia

- b12 IOC_Irq: Interrupt on complete
1 se ha completado una transferencia, si además el bit IOC_IrqEn=1 el DMA genera una int.

IMPORTANTE: Hay que escribir 1 para limpiar el bit

3. Kria-test

Código pynq para testear Resize y DMA

a) Funciones pynq para leer los registros de status (SR) y control (CR)

```
MM2S_DMACR=dma_data.mmio.read(0x00) # reset,reserv,RS
MM2S_DMASR=dma_data.mmio.read(0x04) # idle, Halted
MM2S_DMASourceAdd=dma_data.mmio.read(0x18)
MM2S_DMALength=dma_data.mmio.read(0x28)
S2MM_DMACR=dma_data.mmio.read(0x30) # reset,reserv,RS
S2MM_DMASR=dma_data.mmio.read(0x34) # idle, Halted
S2MM_DMASourceAdd=dma_data.mmio.read(0x48)
S2MM_DMALength=dma_data.mmio.read(0x58)
```

DMA_TYPE_RX => S2MM offset (0x30)

DMA_TYPE_TX => MM2S offset (0x00)

b) Código de test (con los resultados de los print y código de las lib de pynq)

```
# 1) Obtenemos los manejadores de los DMA
dma_cfg = overlay.axi_dma_cfg
dma_data = overlay.axi_dma_data
#PYNQ
# 1) Configura el offset de acceso a los registros (0x00 o 0x18)
# 2) Si el channel es TX (MM2S) prepara el flush
----- Estado inicial-----
MM2S_DMACR=10003      # IOC_IrqEn(0), reset (0),reserv (1), RS(1)
MM2S_DMASR=0          # idle (0), Halted(0)
MM2S_DMASourceAdd=0
MM2S_DMALength=0
S2MM_DMACR=10003
S2MM_DMASR=0
S2MM_DMASourceAdd=0
S2MM_DMALength=0
-----

# 2) Arrancamos los DMA
dma_cfg.sendchannel.start()
dma_data.sendchannel.start()
dma_data.recvchannel.start()

#PYNQ
# 1) Escribe DMACR.RS=1
# 2) Opcional. Si int. habilitadas escribe DMACR.IOC_IrqEn=1
```

```

def start(self):
    """Start the DMA engine if stopped"""
    if self._interrupt:
        self._mmio.write(self._offset, 0x1001) # CR<= IOC_IrqEn(1), reset (0), RS(1)
    else:
        self._mmio.write(self._offset, 0x0001) # CR<= IOC_IrqEn(0), reset (0), RS(1)
    while not self.running:
        pass
    self._first_transfer = True                # _first_transfer=true

```

```

---Estado después de start channels-----
MM2S_DMACR=10003 # IOC_IrqEn(0), reset (0),reserv (1), RS(1)
MM2S_DMASR=0
MM2S_DMASourceAdd=0
MM2S_DMALength=0
S2MM_DMACR=10003 # IOC_IrqEn(0), reset (0),reserv (1), RS(1)
S2MM_DMASR=0
S2MM_DMASourceAdd=0
S2MM_DMALength=0
-----

```

```

# 3) Obtiene los punteros e inicializa
# Obtiene puntero al buffer de datos entrada
src_buf = allocate(shape=(height*width), dtype=np.uint8,
cacheable=1)

```

No encuentro la definición de allocate en la lib de pyng

```

# - inicializa el buffer de data
src_buf[:] = img_gray.ravel()

# Calcula los parámetros de configuración
scale = pow(1.2,1) #pow(1.2, 1) #pow(1.0,1)
new_width = int(width/scale)
new_height = int(height/scale)

```

```

# Obtiene los punteros a los buffer de datos salida y
configuración
des_buf = allocate(shape=((new_height*new_width)),
dtype=np.uint8, cacheable=1)

```

No encuentro la definición de allocate en la lib de pyng

```

cfg_src_buf = allocate(shape=(4, ), dtype=np.uint32)
No encuentro la definición de allocate en la lib de pyng

```

```

# - inicializa el buffer cfg
cfg_src_buf[0] = width
cfg_src_buf[1] = height
cfg_src_buf[2] = scale * pow(2, 14) # 1.2*2^14=19660
cfg_src_buf[3] = 1 / scale * pow(2, 14) # (1/1.2)*2^14=13653

```

```

# 3) Realizamos las transferencias
---Estado antes de transferencia-----
MM2S_DMACR=10003
MM2S_DMASR=0

```

```

MM2S_DMASourceAdd=0
MM2S_DMALength=0
S2MM_DMACR=10003
S2MM_DMASR=0
S2MM_DMASourceAdd=0
S2MM_DMALength=0
-----
dma_cfg.sendchannel.transfer(cfg_src_buf)
dma_data.sendchannel.transfer(src_buf)
dma_data.recvchannel.transfer(des_buf)

#PYNQ
# 1) Verifica que: DMASR.Halt=0, DMASR.Idle=1, datos alineados.
# Si no ERROR
# 2) Si el channel es tipo TX (MM2S) hace flush()
# 3) Escribe los reg. MM2S_SA y MM2S_SA_MSB / S2MM_DA y S2MM_DA_MSB
# 4) Escribe el reg. XX_LENGTH para inciar la transferencia

def running(self): #devuelve true si el bit DMASR.Halt=0 (channel running)
    """True if the DMA engine is currently running"""
    return self._mmio.read(self._offset + 4) & 0x01 == 0x00
def idle(self): #devuelve true si el bit DMASR.Idle=1 (DMA paused)
    """True if the DMA engine is idle
    `transfer` can only be called when the DMA is idle
    """
    return self._mmio.read(self._offset + 4) & 0x02 == 0x02

def transfer(self, array, start=0, nbytes=0):
# array= An contiguously allocated array to be transferred
# start= offsett into array to start
# nbytes= n of bytes to transfer
    if not self.running:
        raise RuntimeError("DMA channel not started")
    if not self.idle and not self._first_transfer:
        raise RuntimeError("DMA channel not idle")
    if nbytes == 0:
        nbytes = array.nbytes - start
    if nbytes > self._max_size:
        raise ValueError(
            "Transfer size is {} bytes, which exceeds "
            "the maximum DMA buffer size {}".format(nbytes, self._max_size))

    if not self._dre and ((array.physical_address+start)%self._align)!=0:
        raise RuntimeError("#si data re-alignment is not enabled (dre) y nbytes a
transferir no es un múltiplo del tamaño de los datos (_align) => error
            "DMA does not support unaligned transfers; "
            "Starting address must be aligned to "
            "{} bytes.".format(self._align) )
    if self._flush_before: # si el channel es tipo TX (MM2S) hacer flush
        array.flush()
        self.transferred = 0 # inicia transferencia
    self._mmio.write(self._offset + 0x18, (array.physical_address +
start) & 0xFFFFFFFF) # Escribe SA / DA
    self._mmio.write(self._offset + 0x1C, ((array.physical_address +
start) >> 32) & 0xFFFFFFFF ) # Escribe SA_MSB / DA_MSB
    self._mmio.write(self._offset + 0x28, nbytes) # Escribe XX_LENGTH
    self._active_buffer = array
    self._first_transfer = False

```

```
dma_cfg.sendchannel.wait()
dma_data.sendchannel.wait()
dma_data.recvchannel.wait()
```

```
#PYNQ
```

```
# 1) Verifica que el channel se arrancó: DMASR.Halt=0
```

```
# 2) Verifica continuamente el registro DMACR hasta que
DMACR.Idle=0 (hasta que haya finalizado la transferencia y DMA
esté en pause)
```

```
# 3) Si el canal es tipo RX (S2MM) hace invalidate()
```

```
# 4) Lee el reg. XX_LENGTH (n bytes transferidos)
```

```
def wait(self):
    """Wait for the transfer to complete"""
    if not self.running:
        raise RuntimeError("DMA channel not started")
    while True:
        error = self._mmio.read(self._offset + 4) # Lee SR
        if self.error: # Si error decodifica el tipo de error
            if error & 0x10:
                raise RuntimeError("DMA Internal Error (transfer length
                0?)")
            if error & 0x20:
                raise RuntimeError("DMA Slave Error (cannot access memory
                map interface)")
            if error & 0x40:
                raise RuntimeError("DMA Decode Error (invalid address)")
        if self.idle:
            break
    if not self._flush_before: # si canal tipo S2MM invalidate
        self._active_buffer.invalidate()
    self.transferred = self._mmio.read(self._offset + 0x28)
```

```
---Estado después de transferencia----
```

```
MM2S_DMCR=10003
MM2S_DMASR=1002
MM2S_DMASourceAdd=37000000
MM2S_DMALength=4B000
S2MM_DMCR=10003
S2MM_DMASR=1002
S2MM_DMASourceAdd=361C0000
S2MM_DMALength=2418C
bytes_read=147852
-----
```

4. Puntos a verificar

a) flush / invalidate

- antes de la transferencia si el canal es MM2S hace un flush del buffer
- después de la transferencia si el canal es S2MM hace un invalidate del buffer

b) Para DMA con interrupciones habilitadas (se habilitan durante start) existe un wait_async Verificar como se limpia el bit de int. (SR.IOC_Irq=1)

```
#PYNQ
# 1) Verifica que el channel se arrancó: DMASR.Halt=0
# 2) Queda esperando la interrupción
# (ver código concurrente utilizando la sintaxis async/await.)
# 3) Limpia la interrupción #Escribe SR.IOC_Irq=1
# 4) Si el canal es tipo TX (S2MM) realiza invalidate()
# 5) Lee el reg. XX_LENGTH (n bytes transferidos)

def _clear_interrupt(self):
    self._mmio.write(self._offset + 4, 0x1000) #Escribe CR.IOC_Irq=1

async def wait_async(self):
    """Wait for the transfer to complete"""
    if not self.running:
        raise RuntimeError("DMA channel not started")
    while not self.idle:
        await self._interrupt.wait()
    self._clear_interrupt()
    if not self._flush_before:
        self._active_buffer.invalidate()
    self.transferred = self._mmio.read(self._offset + 0x28)
```

/pynq/buffer.py

```
class PynqBuffer(np.ndarray):
    """A subclass of numpy.ndarray which is allocated using
    physically contiguous memory for use with DMA engines and
    hardware accelerators. As physically contiguous memory is a
    limited resource it is strongly recommended to free the
    underlying buffer with `close` when the buffer is no longer
    needed. Alternatively a `with` statement can be used to
    automatically free the memory at the end of the scope.

    This class should not be constructed directly and instead
    created using `pynq.allocate()`.

    Attributes
    -----
    device_address: int
        The physical address to the array
    coherent: bool
        Whether the buffer is coherent

    """
```

```

    def __new__( cls, *args, device=None, device_address=0, bo=0,
coherent=False, **kwargs ):
        self = super().__new__(cls, *args, **kwargs)
        self.device_address = device_address
        self.coherent = coherent
        self.bo = bo
        self.device = device
        self.offset = 0
        self.freed = False
        return self

    def flush(self):
        """Flush the underlying memory if necessary"""
        if not self.coherent:
            self.device.flush(self.bo, self.offset,
self.virtual_address, self.nbytes)

    def invalidate(self):
        """Invalidate the underlying memory if necessary"""
        if not self.coherent:
            self.device.invalidate(self.bo, self.offset,
self.virtual_address, self.nbytes )

```